

Übungsblatt 8: Multicore-Synchronisation und Safety

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Laborübung verwandeln Sie Ihren Code in eine ausfallsichere Serien-Architektur. Sie ersetzen das naive „Ping-Pong“-Protokoll aus Termin 6 durch echte Hardware-Semaphoren (**HSEM**), um asynchrone Multicore-Zugriffe auf der Ebene der Bus-Matrix physikalisch abzusichern. Im zweiten Teil implementieren Sie Safety-Mechanismen: Sie überwachen das Timing der FreeRTOS-Task mit einem **Window Watchdog (WWDG)** und programmieren einen Safe Shutdown über den **Early Wakeup Interrupt (EWI)**.

Aufgabe 1: Asynchrone Multicore-Synchronisation (HSEM)

Wir entfernen die alten Handshake-Flags (`m7_ready`, `m4_ready`) und lassen beide Kerne asynchron und so schnell wie möglich denselben Zähler im Shared Memory erhöhen. Nutzen Sie weiterhin die Shared-Variable `sd` aus `shared_data.h` (Feld `payload_value`, analog zu Übungsblatt 6).

1. Öffnen Sie Ihr Dual-Core-Projekt aus Woche 6 im CubeMX Configurator.
2. Aktivieren Sie unter *System Core* das Modul **HSEM** und weisen Sie es **beiden** Kernen (Cortex-M7 und M4) zu. Generieren Sie den Code.
3. Entfernen Sie in beiden `main.c` / `freertos.c`-Dateien die alten `while (sd.m7_ready == 0)`- bzw. `while (sd.m4_ready == 0)`-Schleifen.
4. Implementieren Sie auf beiden Kernen den Zugriff auf `sd.payload_value++`, abgesichert durch `HAL_HSEM_FastTake(0)` und `HAL_HSEM_Release(0, 0)`.
5. Denken Sie daran: Der M4 darf im FreeRTOS-Kontext bei einem belegten Schloss **nicht** durch eine leere `while`-Schleife (Spinlock) blockieren. Fügen Sie hier ein `osDelay(1)` ein. (*Labor-Kompromiss: In der Vorlesung wurde der elegantere HSEM-Interrupt-Pfad vorgestellt. Hier reicht die pragmatische Stufe mit `osDelay(1)`.*)

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Das HSEM garantiert die Atomarität über Core-Grenzen hinweg (HAL_HSEM_FastTake / HAL_HSEM_Release). FastTake ist ein Try-Lock: Es blockiert nicht und liefert sofort HAL_OK oder HAL_ERROR.

Cortex-M4 (FreeRTOS) — bei belegtem Schloss die CPU über osDelay(1) abgeben:

```
for (;;) {
    /* 1. Hardware-Schloss (ID 0) anfordern */
    while (HAL_HSEM_FastTake(0) != HAL_OK) {
        osDelay(1); /* CPU abgeben, wenn Schloss belegt */
    }

    /* 2. --- CRITICAL SECTION --- */
    sd.payload_value++;
    /* ----- */

    /* 3. Hardware-Schloss freigeben (Process ID 0) */
    HAL_HSEM_Release(0, 0);
}
```

Cortex-M7 (Bare-Metal) — gleiche Critical Section, aber ohne osDelay (klassisches Polling; kein RTOS, das blockiert werden könnte):

```
for (;;) {
    /* 1. Hardware-Schloss (ID 0) anfordern - Polling ohne OS */
    while (HAL_HSEM_FastTake(0) != HAL_OK) {
        /* busy wait / kurzes Polling ausreichend */
    }

    /* 2. --- CRITICAL SECTION --- */
    sd.payload_value++;
    /* ----- */

    /* 3. Hardware-Schloss freigeben (Process ID 0) */
    HAL_HSEM_Release(0, 0);
}
```

Aufgabe 2: Den Window Watchdog (WWDG) integrieren

Wir härten das System nun gegen Software-Hänger und fehlerhafte (zu schnelle) Ausführungen ab.

1. Öffnen Sie CubeMX und aktivieren Sie den **WWDG** für den Cortex-M4.
2. Konfigurieren Sie den Watchdog:

- *WWDG clock prescaler*: 8
 - *Window value*: 80 (das erlaubte Fenster beginnt erst bei Zählerstand 80)
 - *Free-running downcounter value*: 120 (Startwert)
3. Aktivieren Sie im Reiter *NVIC Settings* zwingend den **Early Wakeup Interrupt (EWI)**. Generieren Sie den Code.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Die Konfiguration erzeugt ein striktes zeitliches Fenster (*Window*):

Phase	Zählerstand (Beispiel)	Refresh erlaubt?	Folge bei Refresh / Ablauf
Early Phase	120 ... 81	nein	sofortiger Reset
Allowed Window	80 ... > 0	ja	Zähler wird neu geladen
Late Phase	unter 0x40 (Reset)	zu spät	Reset, Deadline verpasst

Wenn der Zähler von 120 abwärts läuft, ist das Füttern (`HAL_WWDG_Refresh`) in der Early Phase streng verboten. Erst im Allowed Window (ab Wert 80) darf das System refreshen. Der EWI liefert zudem bei Zählerstand 0x40 den maskierbaren Interrupt `WWDG_IRQn` (typisch NVIC-Prio 0) — die letzte Chance für einen Safe Shutdown, bevor der Hardware-Reset beim Übergang 0x40→0x3F zuschlägt.

Hinweis zur Clock: Die Fensterbreite in Millisekunden hängt von f_{PCLK1} ab (WWDG an APB1). Bei typischem Nucleo-Setup (≈ 100 MHz) und Prescaler 8 liegen `osDelay(15)` / `osDelay(50)` / `osDelay(5)` im erlaubten bzw. Late-/Early-Bereich — prüfen Sie die Werte bei abweichender Clock-Konfiguration nach.

Aufgabe 3: Watchdog füttern und Crash-Tests

Kalibrieren Sie das Timing Ihrer Task und testen Sie das Sicherheitsnetz.

1. Fügen Sie in die Endlosschleife Ihrer M4-Task ein `osDelay(15)` ein. Rufen Sie danach `HAL_WWDG_Refresh(&hwwdg)` auf. Das System sollte nun stabil laufen.
2. **Crash-Test 1 (Late Phase):** Ändern Sie das Delay auf `osDelay(50)`. Flashen Sie den Code und erklären Sie, was passiert.
3. **Crash-Test 2 (Early Phase):** Ändern Sie das Delay auf `osDelay(5)`. Flashen Sie den Code und erklären Sie, was nun passiert und warum dieses Verhalten für die Safety wichtig ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Stabiler Betrieb (Refresh im Allowed Window):

```
void StartDefaultTask(void *argument)
{
    for (;;) {
        /* ... HSEM-Logik (Aufgabe 1) ... */

        osDelay(15);           /* Timing ins Allowed Window */
        HAL_WWDG_Refresh(&hwwdg); /* füttern */
    }
}
```

Crash-Test 1 — Late Phase (osDelay(50)):

Der Refresh kommt zu spät. Der Zähler unterschreitet 0x40 (Late Phase). Die WWDG-Hardware zieht die Reset-Leitung; das Board startet neu. Damit werden klassische Deadlocks und „hängende“ Tasks abgefangen.

Crash-Test 2 — Early Phase (osDelay(5)):

Die Task füttert den Watchdog zu früh (Early Phase, z. B. Zähler noch bei ≈ 110). Die Hardware löst **sofort** einen Reset aus — auch ohne dass die Deadline erreicht wurde. Das ist essenziell für die Safety: Es verhindert, dass Pointer-Bugs oder Bit-Flips (z. B. kosmische Strahlung) wichtige Validierungen überspringen und der Code unzulässig „zu schnell“ abläuft.

Aufgabe 4: Safe Shutdown via Early Wakeup Interrupt (EWI)

Bevor der WWDG das System beim Absturz hart neu startet, wollen wir das System in einen sicheren Zustand (*Safe State*) überführen.

1. Überschreiben Sie in der `main.c` des Cortex-M4 die Funktion `HAL_WWDG_EarlyWakeupCallback(WWDG_Handle_t *hwwdg)`.
2. Schalten Sie innerhalb dieses Callbacks die **rote** Fehler-LED des Boards ein (Nucleo-H755ZI-Q: **LD3 an PB14**, nicht die gelbe LD2 an PE1).
3. Führen Sie Crash-Test 1 (Late Phase) erneut aus. Was beobachten Sie an der roten LED? Darf der Watchdog in diesem Callback noch gefüttert werden?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

```
void HAL_WWDG_EarlyWakeupCallback(WWDG_HandleTypeDef *hwwdg)
{
    /* Die letzten Mikrosekunden vor dem Reset */
    /* LD3 rot (PB14) einschalten, um den EWI sichtbar zu machen */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET);

    /* HIER NIEMALS REFRESH AUFRUFEN!
     * Das System ist korrupt und bootet gleich hart neu. */
}
```

Beobachtung: Kurz bevor das Board durch den provozierten Crash (Late Phase) neu startet, blitzt die rote LD3 extrem kurz auf — der EWI war live sichtbar.

Regel: Der Watchdog darf in diesem Callback **niemals** mehr gefüttert werden. Wenn der EWI auslöst, liegt bereits ein fataler Timing-Fehler vor. Der Sinn des EWI ist ausschließlich Schadensbegrenzung (z. B. Motor-PWMs abschalten, Fehlercode in den Backup-SRAM schreiben), bevor der unvermeidliche Hardware-Reset wenige Taktzyklen später zuschlägt.